

Object-oriented Programming

Week 3 | Lecture 1,2 and 3

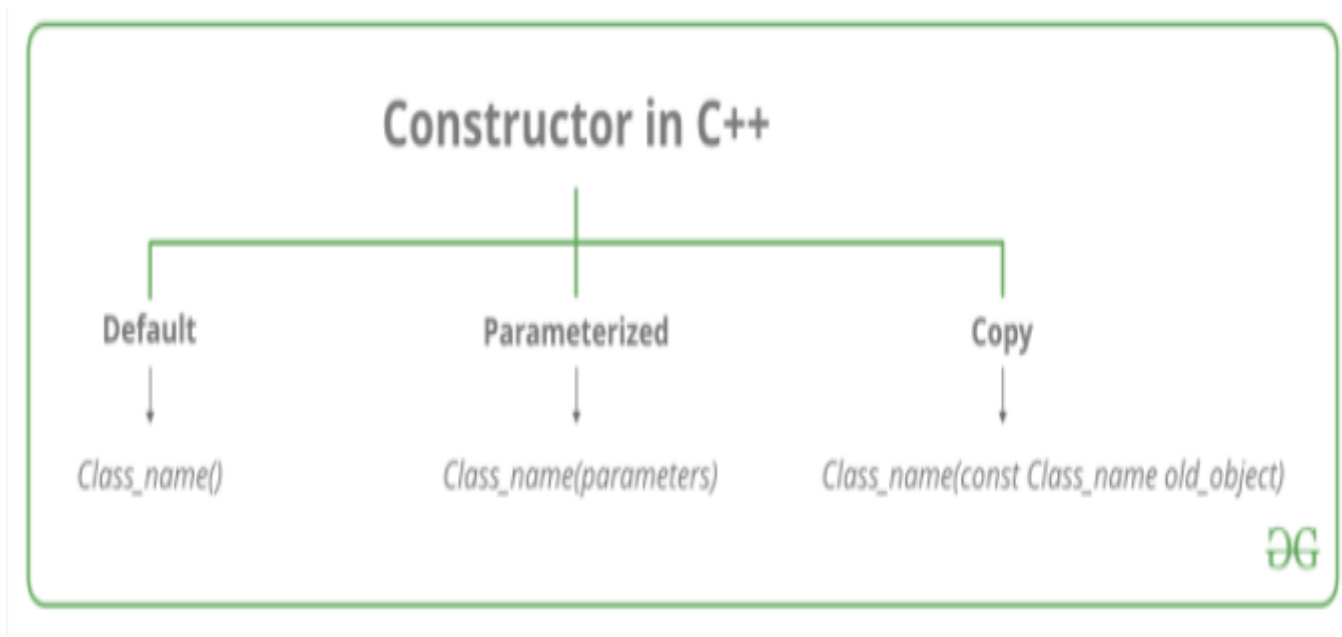
Constructor

- A *constructor* is called whenever an object of a class is created
- A constructor is a member function of a class which initializes objects of a class. In C++, Constructor is automatically called when object(instance of class) create.

How constructors are different from a normal member function?

- Constructor has same name as the class itself
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).

Types of constructors



Default Constructor

- Default constructor is the constructor which doesn't take any argument. It has no parameters.
- Even if we do not define any constructor explicitly, the compiler will automatically provide a default constructor implicitly.

Default Constructor

```
class MyClass
```

```
{
```

```
    public: //constructors are usually public
```

```
    MyClass()
```

```
    { //Things you want to do as soon as object is  
      created }
```

```
}
```

Constructor

- C++ requires a constructor call for each object that is created
- A constructor **looks** similar to a function except that it does not return anything (thus it does not have a return type)

Constructor

- A constructor is used to initialize the data members/class variables for an object when that object is created
- If you don't define a constructor in a class, the compiler itself provides a *default (no-parameter) constructor* for that class

Example code

```
#include <iostream>
```

```
using namespace std;
```

```
class Line {  
    public:  
        void setLength( double len );  
        double getLength( void );  
        Line(); // This is the constructor  
    private:  
        double length;  
};
```

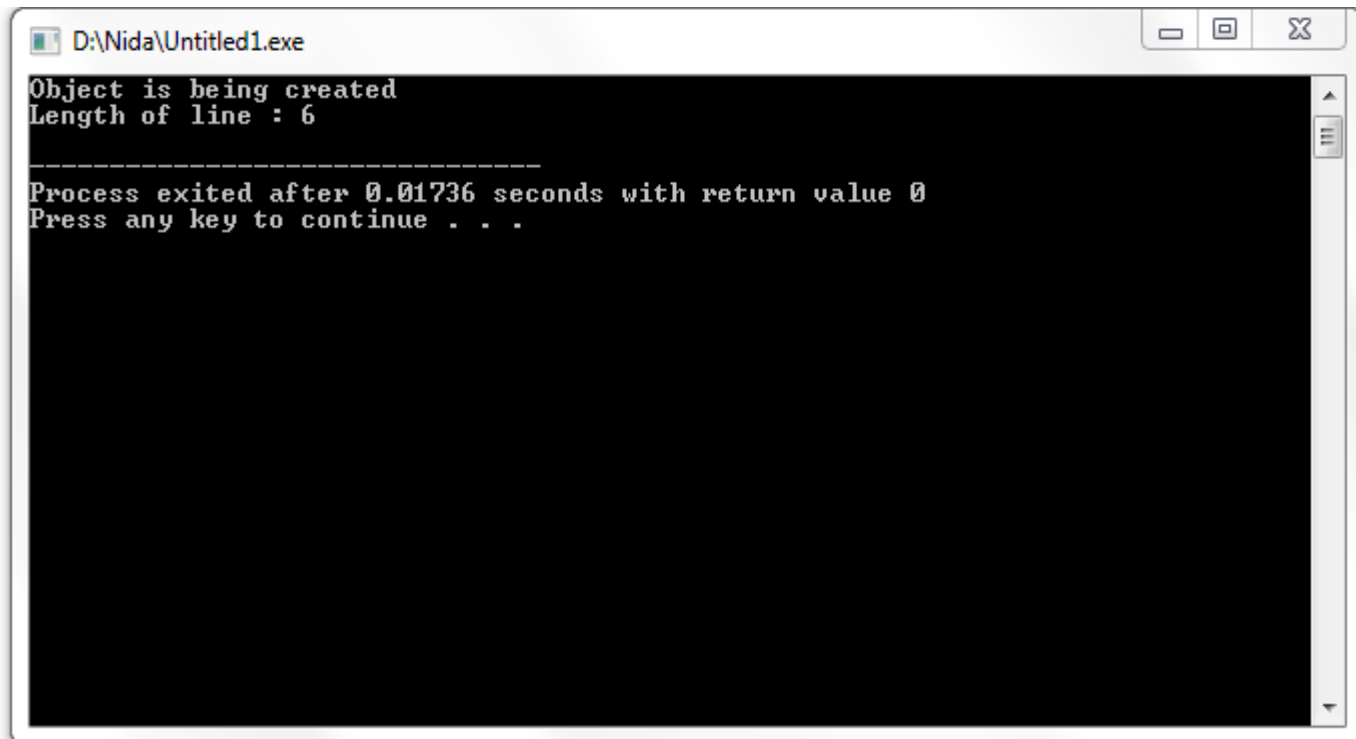
Member functions definitions including constructor

```
Line::Line(void) {  
    cout << "Object is being created" << endl;  
}  
void Line::setLength( double len ) {  
    length = len;  
}  
double Line::getLength( void ) {  
    return length;  
}
```

Main function for the program

```
int main() {  
    Line line;  
  
    // set line length  
    line.setLength(6.0);  
    cout << "Length of line : " << line.getLength()  
    << endl;  
  
    return 0;  
}
```

it produces the following result



```
D:\Nida\Untitled1.exe
Object is being created
Length of line : 6
-----
Process exited after 0.01736 seconds with return value 0
Press any key to continue . . .
```

Parameterized Constructors:

- A constructor can take parameters. These parameters are used to initialize class variables for the object

```
class Employee
{
    string name;

    public:
    Employee(string eName)
    {
        name = eName;
    }
}
```

```
int main()
{
    Employee e1("Ali");
    Employee e2("Shuja");
}
```

Constructor

- Can we have more than one parameterized constructors of a class?

Yes, we can. But not having the same parameter signature

- If you define any parameterized constructor(s) in the class, C++ will not implicitly create a default constructor for you

Some Code

```
class A
{
    int var;
    string str;

    public:
    A(int v)
    {
        var = v;
        cout << "Constructor v1: " << var << endl;
    }

    A()
    {
        var = 0;
        cout << "Default constructor: " << var << endl;
    }

    A(string s)
    {
        str = s;
        cout << "Constructor v2: " << str << endl;
    }
};
```

```
int main()
{
    A dOb;
    A iOb(50);
    A sOb("Hello");
}
```

Some Code

```
class A
{
    int var;
    string str;

    public:
    A(int v)
    {
        var = v;
        cout << "Constructor v1: " << var << endl;
    }

    A(string s)
    {
        str = s;
        cout << "Constructor v2: " << str << endl;
    }
};
```

```
int main()
{
    A dOb;    \\This line will give an error
    A iOb(50);
    A sOb("Hello");
}
```


Default Arguments in C++

- A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn't provide a value for the argument with a default value.
- Default arguments are overwritten when calling function provides values for them.

```
1  #include <iostream>
2  using namespace std;
3  class test{
4      int a;
5      public:
6          test(int x = 100){
7              a=x;
8          }
9          int gettest(){
10             return a;
11         }
12 };
13 main(){
14     test t;
15     cout<< t.gettest();|
16
17 }
```

 C:\Users\Nida Munawar\Downloads\Untitled1.exe

100

Process exited after 0.1747 seconds with return value 0

Press any key to continue . . .

output of the following C++ code?

```
#include <iostream>
#include <string>
using namespace std;
class A{
    int a;
public:
    A(int i){
        a = i;
    }
    void assign(int i){
        a = i;
    }
    int return_value(){
        return a;
    }
};
int main()
{
    A obj;
    obj.assign(5);
    cout<<obj.return_value();
}
```

Output

- Error

output of the following C++ code?

```
#include <iostream>
#include <string>
using namespace std;
class A{
    int a;
public:
    A(int i= 100){
        a = i;
    }
    void assign(int i){
        a = i;
    }
    int return_value(){
        return a;
    }
};
int main()
{
    A obj;
    obj.assign(5);
    cout<<obj.return_value();
}
```

output of the following C++ code?

```
#include <iostream>
#include <string>
using namespace std;
class A{
    int a;
public:
    A(int i= 100){
        a = i;
    }
    A(){
    }
    void assign(int i){
        a = i;
    }
    int return_value(){
        return a;
    }
};
int main()
{
    A obj;
    obj.assign(5);
    cout<<obj.return_value();
}
```

Output

[Error] call of overloaded 'A()' is ambiguous

Rules for Default Arguments

1. Default values must be assigned from right to left.

- Wrong:

```
void test(int a = 5, int b); // ERROR: 'b' must have a default if 'a' has one
```

- Correct:

```
void test(int a, int b = 10);
```

2. Default arguments can only be specified once (either in function declaration or definition).

- Correct:

```
void show(int x = 100); // Default argument in declaration
```

```
void show(int x) { // Definition without default value std::cout << x; }
```

- Wrong:

```
void show(int x = 100); void show(int x = 200); // ERROR: Default argument redefined
```

3. Default arguments can be mixed with non-default ones, but non-default parameters must come first.

- Correct:

```
void print(int a, int b = 20, int c = 30);
```

- Wrong:

```
void print(int a = 10, int b, int c = 30); // ERROR
```

Anonymous classes in C++

- Anonymous class is a class which has no name given to it
- Anonymous classes cannot have explicit constructors or destructors, the compiler provides a **default constructor**.
- These classes can neither be passed as arguments to functions nor can be used as return values from functions.

Anonymous classes in C++

```
#include <iostream>
using namespace std;
class
{
    int i;
public:
    void setData(int a) {
        i = a;
    }
    void print(){
        cout << "Value for i : " << this->i << endl;
    }

} obj1, obj2;    // multiple objects for anonymous class

// Driver function
int main() {
    obj1.setData(10);
    obj1.print();

    obj2.setData(20);
    obj2.print();
    return 0;
}
```

this POINTER

1. Each object gets its own copy of the data member.
 2. All-access the same function definition as present in the code segment.
- Meaning each object gets its own copy of data members and all objects share a single copy of member functions.
- Then now question is that if only one copy of each member function exists and is used by multiple objects, how are the proper data members are accessed and updated?

this POINTER

```
#include <iostream>
using namespace std;
class car{
    int a;
    public:
    car(int e){
        a=e;}

    void set(int e){
        a=e; //you always see the data member which belong to particular object it never mixed up
    };

    int main(){
        car c(10);
        car c1(12);
        c.set(22); // when you call some member function using some object
        c1.set(32);
    }
```

this POINTER

- You can access class members using By default, the compiler provides each member function of a class with an implicit parameter that points to the object through which the member function is called. The implicit parameter is this pointer.

This pointer holds the address of current/ calling object

```
#include <iostream>
using namespace std;
class car{
    int a;
    public:
    car(int e){
        a=e;}

    void set(int e){
        this -> a=e; // this is how compiler internally achieve this
                    };

    int main(){
        car c(10);
        car c1(12);
        c.set(22); // when you call some member function using some object
        c1.set(32);
    }
```

Note

When you call a non-static member function on an object, the compiler implicitly passes the address of the calling object (this) to the function.

For example, the following two calls are equivalent:

```
obj.show(); // Implicitly: show(&obj)
```

```
obj.setValue(5); // Implicitly: setValue(&obj, 5)
```

Internally, the compiler transforms:

```
class Example {  
public:  
void show() {  
    std::cout << "Hello" << std::endl; } };
```

Into something like this:

```
class Example {  
public:  
void show(Example* this) {  
    // Compiler implicitly passes 'this'  
    std::cout << "Hello" << std::endl; } };
```

Does the Compiler Pass this to Constructors?

- Yes, the compiler always passes this to non-static member functions, including constructors.
- You don't need to write this explicitly unless there is a name conflict (e.g., between constructor parameters and member variables).
- this is not required in static functions because they don't belong to any specific instance.

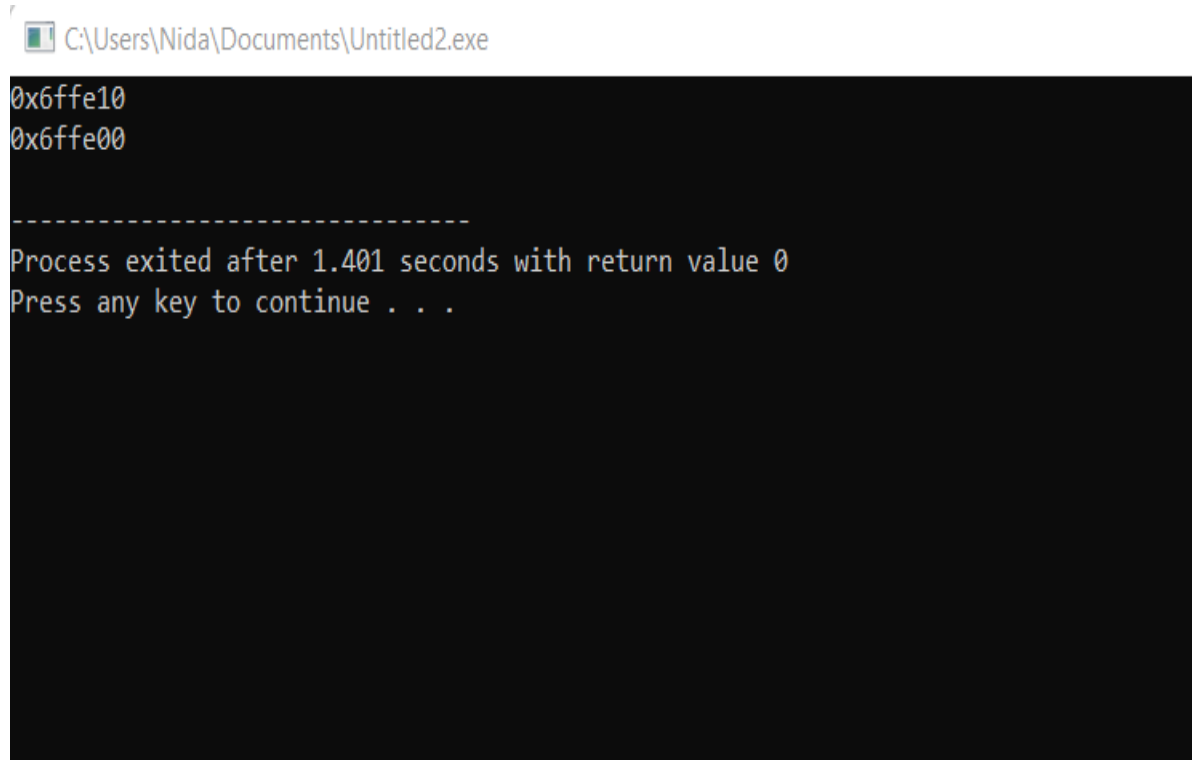
1. usage of this pointer

Print address of object inside class

```
#include <iostream>
using namespace std;
class car{
    int a;
    public:
        void set(int e){
            a=e;    }
        void printAddress(){
            cout << this << endl;
        }
};

int main(){
    car obj1;
    car obj2;
    obj1.printAddress(); // address of obj1
    obj2.printAddress();// address of obj2
}
```

Print address of object inside class



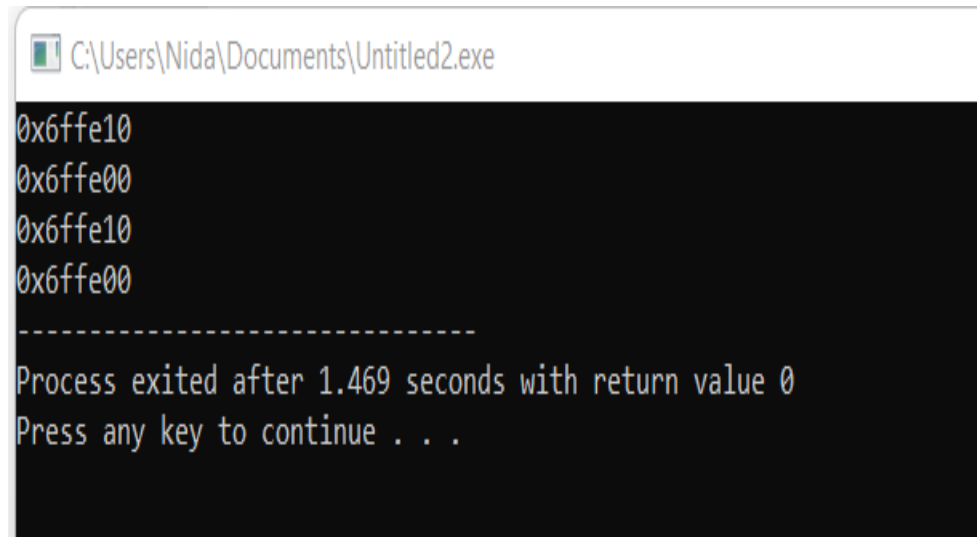
```
C:\Users\Nida\Documents\Untitled2.exe  
0x6ffe10  
0x6ffe00  
-----  
Process exited after 1.401 seconds with return value 0  
Press any key to continue . . .
```

Print address of object inside and outside the class

```
#include <iostream>
using namespace std;
class car{
    int a;
    public:
        void set(int e){
            a=e;        }
        void printAddress(){
            cout << this << endl;
        }
};

int main(){
    car obj1;
    car obj2;
    obj1.printAddress(); // address of obj1
    obj2.printAddress();// address of obj2
    cout << &obj1 << endl;// address of obj1
    cout << &obj2;// address of obj2
}
```

Print address of object inside and outside the class



```
C:\Users\Nida\Documents\Untitled2.exe
0x6ffe10
0x6ffe00
0x6ffe10
0x6ffe00
-----
Process exited after 1.469 seconds with return value 0
Press any key to continue . . .
```

2. usage of this pointer

When local variable's name is same as member's name

```
#include <iostream>
using namespace std;
class example{
private:
int x; public:
void set(int x){
x = x;}
void printAddressAndValue(){
cout<< "the value is "<<x<<endl;}};
main(){
    example e;
    e.set(3);
    e.printAddressAndValue();}
```

When local variable's name is same as member's name

```
#include <iostream>
using namespace std;
class example{
private:
int x; public:
void set(int x){
(*this).x = x;} // The 'this' pointer is used to retrieve the object's x
// hidden by the local variable 'x'
void printAddressAndValue({
cout<<"The address is "<<this<<" and the value is "<<(*this).x<<endl;});
main(){
    example e;
    e.set(3);
    e.printAddressAndValue();}
```


this POINTER

- `#include <iostream>`
- `using namespace std;`
- `class example{`
- `private:`
- `int x; public:`
- `void set(int x){`
- `this->x = x;}`
- `void printAddressAndValue(){`
- `cout<<"The address is "<<this<<" and the value is "<<this->x <<endl;`
- `}`
- `};`
- `main(){`
- `example e;`
- `e.set(3);`
- `e.printAddressAndValue();}`

Next Lecture

- Destructors!

“Prepare to Die”



Destructor

- A class's destructor is automatically called when an object of that class is “**destroyed**”
- **Destruction** of an object means when program execution leaves the scope in which object was instantiated
- Destructors are a “prepare to die” member function. They are often abbreviated “dtor”.

Destructor

- We can declare a destructor by adding a tilde (~) as prefix to its class name.
- Has the same name as that of class

Example: `~MyClass() { . . . }`

Destructor

- A destructor cannot return a value and cannot take any arguments
- A destructor cannot be overloaded
- A class can thus have only one destructor
- If you do not explicitly define a destructor, the compiler provides a default “empty” destructor

Order of calling Destructors

```
class MyClass
{
    int objectID;

    MyClass(int objectID1)
    {
        objectID = objectID1;
    }

    ~MyClass()
    {
        cout << objectID << " deleted";
    }
}
```

```
#include <iostream>
using namespace std;
class ABC{
    public:
        ABC () //constructor defined
        {
            cout << "Hey look I am in constructor" << endl; }
        ~ABC() //destructor defined
        {
            cout << "Hey look I am in destructor" << endl;}};
int main(){
    ABC cc1; //constructor is called
    ABC cc2; //constructor is called
    cout << "function main is terminating...." << endl;
    return 0;
}
```


 C:\Users\Nida Munawar\Documents\Untitled1.exe

```
Hey look I am in constructor  
Hey look I am in constructor  
function main is terminating....  
Hey look I am in destructor  
Hey look I am in destructor
```

```
Process exited after 0.127 seconds with return value 0  
Press any key to continue . . .
```

Order of calling Destructors

- **Destructors** for nonstatic member objects are called in the reverse **order** in which they appear in the class declaration
- When objects store in the stack order will be LIFO

Order of calling Destructors

```
MyClass ob1 (1);
```

```
void func()  
{  
    MyClass ob3 (3);  
    MyClass ob4 (4);  
}
```

```
int main()  
{  
    MyClass ob2 (2);  
    func();  
    MyClass ob5 (5);  
}
```

Order of calling Destructors

```
MyClass ob1 (1);           \\ destroyed fifth

void func()
{
    MyClass ob3 (3);       \\ destroyed second
    MyClass ob4 (4);       \\ destroyed first
}

int main()
{
    MyClass ob2 (2);       \\ destroyed fourth
    func();
    MyClass ob5 (5);       \\ destroyed third
}
```

What's the order that local objects are destroyed?

- In reverse order of construction: First constructed, last destroyed.(LIFO)
- In the following example, b's destructor will be executed first, then a's destructor:
- `void userCode()`
- `{`
- `Fred a;`
- `Fred b;`
- `// ...`
- `}`

What's the order that the objects in an array are destroyed?

- In reverse order of construction: First constructed, last destroyed.
- In the following example, the order for destructors will be a[9], a[8], ..., a[1], a[0]:
- `void userCode()`
- `{`
- `Fred a[10];`
- `// ...`
- `}`
-

Why are Destructors useful?

- Useful for garbage collection
- Garbage-collected languages like JAVA do not have a destructor
 - There is no guarantee of when an object will be destroyed

When do we need to write a user-defined destructor?

- If we do not write our own destructor in class, compiler creates a default destructor for us. The default destructor works fine unless we have dynamically allocated memory or pointer in class. When a class contains a pointer to memory allocated in class, we should write a destructor to release memory before the class instance is destroyed. This must be done to avoid memory leak.

When do we need to write a user-defined destructor?

- only variables allocated on the stack are deallocated automatically when they go out of scope.
- We need to call the destructor if have allocated memory in heap

```
1 int a = 5; // allocated on the stack
2 int* b = new int(5); // allocated dynamically, on the heap. Must be deleted
```

```
#include <iostream>
using namespace std;
class Test{
public:
    Test(){
        cout << "Constructor" << endl;}

    ~Test(){
        cout << "Destructor" << endl;    }

    void display(){
        cout << "Display content" << endl;}
};

int main(){
    Test a;
    a.display();}
```

C:\Users\Nida\Documents\Untitled1.exe

Constructor

Display content

Destructor

Process exited after 9.24 seconds with return value 0

Press any key to continue . . .

Here we do not want to call the destructor explicitly because we have created the object in the stack, not in heap. So, after object goes out of scope destructor will call automatically.

What if we create the object in heap instead of stack?

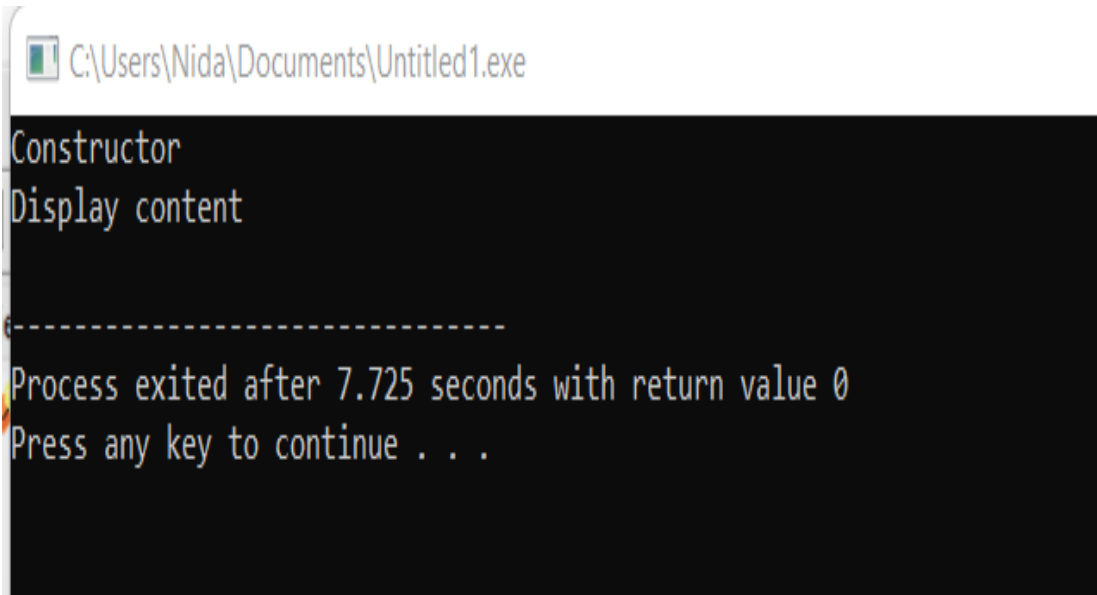
```
#include <iostream>
using namespace std;
class Test{
public:
    Test(){
        cout << "Constructor" << endl;}

    ~Test(){
        cout << "Destructor" << endl;}

    void display(){
        cout << "Display content" << endl;}
};

int main(){
    Test *a = new Test();
    a->display();}
```

What if we create the object in heap instead of stack?



```
C:\Users\Nida\Documents\Untitled1.exe
Constructor
Display content

-----
Process exited after 7.725 seconds with return value 0
Press any key to continue . . .
```

But in this scenario, you can see that it will not call the destructor automatically. So, we have to call it explicitly. Unless memory will not be released at the end of the scope of the object. We can call delete in order to invoke the destructor manually.

Why the Destructor is Not Called:

The destructor is called automatically when an object goes out of scope (for stack-allocated objects) or when delete is explicitly called on a pointer to a dynamically allocated object.

In this case, since delete is not called, the memory is not freed, and the destructor is not executed. This results in a memory leak.

We can call delete in order to invoke the destructor manually.

```
#include <iostream>
using namespace std;
class Test{
    int a;
public:
    Test(){
        a=10;
        cout << "Constructor" << endl;}

    ~Test(){
        cout << "Destructor" << endl;}

    void display(){
        cout << "Display content " << a << endl;}
};

int main(){
    Test *a = new Test();
    a->display();
    delete a;
}
```

We can call delete in order to invoke the destructor manually.

```
#include <iostream>
using namespace std;
class Test{
    int a;
public:
    Test(){
        a=10;
        cout << "Constructor" << endl;}

    ~Test(){
        cout << "Destructor" << endl;}

    void display(){
        cout << "Display content" << a << endl;}
};

int main(){
    Test *a = new Test();
    delete a;
    a->display();
}
```



```
D:\fall 2024\OneDrive\Docum X + v
Constructor
Destructor
Display content12605840

-----
Process exited after 0.2197 seconds with return value 0
Press any key to continue . . . |
```

Another example When do we need to write a user-defined destructor?

```
#include <iostream>
using namespace std;
class ABC
{
    int *p;
public:
    ABC () //constructor defined
    {
        p=new int;
        cout << "inside constructor" << endl;
    }
    ~ABC() //destructor defined
    {
        delete p;
        cout << "inside destructor" << endl;
    }
};
```

When do we need to write a user-defined destructor?

```
int main()
{
    ABC cc1; //constructor is called
    ABC cc2; //constructor is called
    cout << "function main is terminating...." <<
endl;
    /*....object cc1 goes out of scope ,now
destructor is being called...*/
    return 0;
} //end of program
```

Output?

```
#include <iostream>
using namespace std;
class ABC{
    int *p;
public:
    ABC () {
        p=new int;
        cout << "inside constructor" << endl; }
    ~ABC() {
        cout << "inside destructor" << endl;
    };
};
int main(){
    ABC cc1;
    ABC cc2;
    cout << "function main is terminating...." << endl;
    return 0;}
```

What is a memory leak?

- if we continuously allocate memory(dynamically) for a particular object without releasing that memory after completing its task, that memory can not be used in future. Because, although object has completed its task, it is holding that memory because we have not released it properly. So, others cannot grab that memory for their executions.

This wasted memory can be started in few bytes but eventually, end up in megabytes or even more. This might slow down your application in execution when it finds hard to allocate memory or may be crashed as it failed to allocate enough memory for its execution.

So, destructor comes in handy to deallocate memory that it has allocated when the constructor was called.

Exercise

- Write a program to print the names of students by creating a Student class. If no name is passed while creating an object of the Student class, then the name should be "Unknown", otherwise the name should be equal to the String value passed while creating the object of the Student class.(use constructor and destructor)

References in C++

- When a variable is declared as a reference, it becomes an alternative name for an existing variable. A variable can be declared as a reference by putting '&' in the declaration.

```
data_type &ref = variable;
```

use of references

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;

    // ref is a reference to x.
    int& ref = x;

    // Value of x is now changed to 20
    ref = 20;
    cout << "x = " << x << "\n";

    // Value of x is now changed to 30
    x = 30;
    cout << "ref = " << ref << "\n";

    return 0;
}
```


- **Passing By Pointer vs Passing By Reference in C++**

pass by pointer

```
#include <iostream>
using namespace std;
```

```
void swap(int *x, int *y)
{
    int z = *x;
    *x = *y;
    *y = z;
}
```

```
// Driver Code
```

```
int main()
{
    int a = 45, b = 35;
    cout << "Before Swap\n";
    cout << "a = " << a << " b = " << b << "\n";

    swap(&a, &b);

    cout << "After Swap with pass by pointer\n";
    cout << "a = " << a << " b = " << b << "\n";
}
```

Passing By Reference

- It allows a function to modify a variable without having to create a copy of it. We have to declare reference variables. The memory location of the passed variable and parameter is the same and therefore, any change to the parameter reflects in the variable as well.

Modify the passed parameters in a function:

```
#include <iostream>
using namespace std;
```

```
// Function having parameters as
// references
```

```
void swap(int& first, int& second)
```

```
{
```

```
    int temp = first;
```

```
    first = second;
```

```
    second = temp;
```

```
}
```

```
int main()
```

```
{
```

```
int a = 2, b = 3;
```

```
    swap(a, b);
```

```
    // changes can be seen printing both variables
```

```
    cout << a << " " << b;
```

```
    return 0;
```

```
}
```

Next Lecture

- Copy Constructor!

Copy Constructor

- creating a copy of an object means to create an exact replica of the object having the same literal value, data type, and resources.

Syntax

Syntax 1:

Test Obj;
Test Obj1(Obj);

Syntax 2:

Test Obj;
Test Obj1 = Obj;

Syntax 3:

Test Obj;
Test Obj2;
Obj2 = Obj;

Initializing one object with another

```
#include <iostream>
using namespace std;
class A{
    int val;
    public:
    A(int val1) {
        val = val1; }
    void update() { val++; }
    void showVal() { cout << "Value: " << val << endl; }
};
int main()
{
    A a1( 10 );
    a1.showVal( );

    A a2 = a1;

    a2.showVal();
    a2.update();

    a1.showVal( );
    a2.showVal( );

}
```


Initializing one object with another

```
class A{  
    int val;  
    public:  
    A(int val1) {  
        val = val1; }  
    A(){ }  
    void setVal(int val1) { val = val1; }  
    void showVal() { cout << "Value: " << val <<  
endl; }  
};
```

Initializing one object with another

```
int main()
{
    A a1( 10 );
    a1.showVal( );

    A a2 = a1;
    a2.showVal( );
    a2.setVal( 20 );

    a1.showVal( );
    a2.showVal( );

}
```

Initializing one object with another

```
int main()
{
    A a1( 10 );
    a1.showVal( );

    A a2 = a1;
    a2.showVal( );
    a2.setVal( 20 );

    a1.showVal( );
    a2.showVal( );
}
```

OUTPUT:

Val: 10 *a1.val*

Val: 10 *a2.val*

Val: 10 *a1.val*

Val: 20 *a2.val*

Initializing one object with another

```
int main()
{
    A a1( 10 );
    a1.showVal( );

    A a2;
    a2 = a1;
    a2.showVal( );
    a2.setVal( 20 );

    a1.showVal( );
    a2.showVal( );
}
```

OUTPUT:

Val: 10 *a1.val*

Val: 10 *a2.val*

Val: 10 *a1.val*

Val: 20 *a2.val*

Initializing one object with another

```
int main()
{
    A a1( 10 );
    a1.showVal( );

    A a2(a1);
    a2.showVal( );
    a2.setVal( 20 );

    a1.showVal( );
    a2.showVal( );
}
```

OUTPUT:

Val: 10 *a1.val*

Val: 10 *a2.val*

Val: 10 *a1.val*

Val: 20 *a2.val*

Copy Constructor

- A copy constructor is used to initialize an object using another object of the same class
- A copy constructor has the following prototype:

ClassName (const ClassName &ob);

Copy Constructor

- If we don't define our own copy constructor, the compiler creates a default copy constructor for each class
- The default copy constructor performs member-wise copy between objects
- Default copy constructor works fine unless an object has pointers or any runtime allocation

Shallow Copy

- Default constructor always perform a *shallow copy*
- In shallow copy, an object is created by simply copying the data of all variables of the original object.

With new keyword

```
#include <iostream>
using namespace std;
class A{
    int val;
    int *ptr;
    public:
    A(int val1,int p) {
        val = val1;
        ptr = new int;
        *ptr = p;}
    void update() { val++; (*ptr)++;}
    void showVal() { cout << "Value: " << val << " ptr is "
        <<*ptr << endl; }
};
int main()
{
    A a1( 10,20 );
    a1.showVal( );

    A a2 = a1;

    a2.showVal();
    a2.update();

    a1.showVal( );
    a2.showVal( );

}
```

Without new same problem

```
#include <iostream>
using namespace std;
class A {
    int val;
    int *ptr; // Points to an external variable
public:
    A(int val1, int &p) {
        val = val1;
        ptr = &p;
    }

    void update() {
        val++;
        (*ptr)++;
    }

    void showVal() {
        cout << "Value: " << val << ", ptr is " << *ptr << endl;
    }
};

int main() {
    int x = 20;

    A a1(10, x);
    a1.showVal();

    A a2 = a1; // Both share the same x
    a2.showVal();

    a2.update();

    a1.showVal();
    a2.showVal();

    cout << "x value: " << x << endl;

    return 0;
}
```

Reference variable

```
#include <iostream>
using namespace std;

class A {
    int val;
    int &ref; // Reference to an external variable

public:
    A(int val1, int &p) : val(val1), ref(p) {} // Initialize reference in initializer list

    void update() {
        val++;
        ref++; // Directly modify the referenced variable
    }

    void showVal() {
        cout << "Value: " << val << ", ref is " << ref << endl;
    }
};

int main() {
    int x = 20;

    A a1(10, x); // `ref` in a1 now refers to x
    a1.showVal();

    A a2 = a1; // Both `a1` and `a2` refer to the same `x`
    a2.showVal();

    a2.update(); // Updates both val and `x`

    a1.showVal();
    a2.showVal();

    cout << "x value: " << x << endl;

    return 0;
}
```

Another example

- `#include <iostream>`
- `using namespace std;`
- `class box {`
- `int length;`
- `int breadth;`
- `int height;`
- `public:`
- `void set_dimensions(int length1, int breadth1, int height1)`
- `{`
- `length = length1;`
- `breadth = breadth1;`
- `height = height1;`
- `}`

Another example

- `void show_data()`
- `{`
- `cout << " Length = " << length`
- `<< "\n Breadth = " << breadth`
- `<< "\n Height = " << height`
- `<< endl;`
- `}`
- `};`

Another example

- `// Driver Code`
- `int main()`
- `{`
- `box B1, B3;`
-
- `// Set dimensions of Box B1`
- `B1.set_dimensions(14, 12, 16);`
- `B1.show_data();`
- `box B2 = B1;`
- `B2.show_data();`
- `B3 = B1;`
- `B3.show_data();`
-
- `return 0;`
- `}`

Shallow Copy

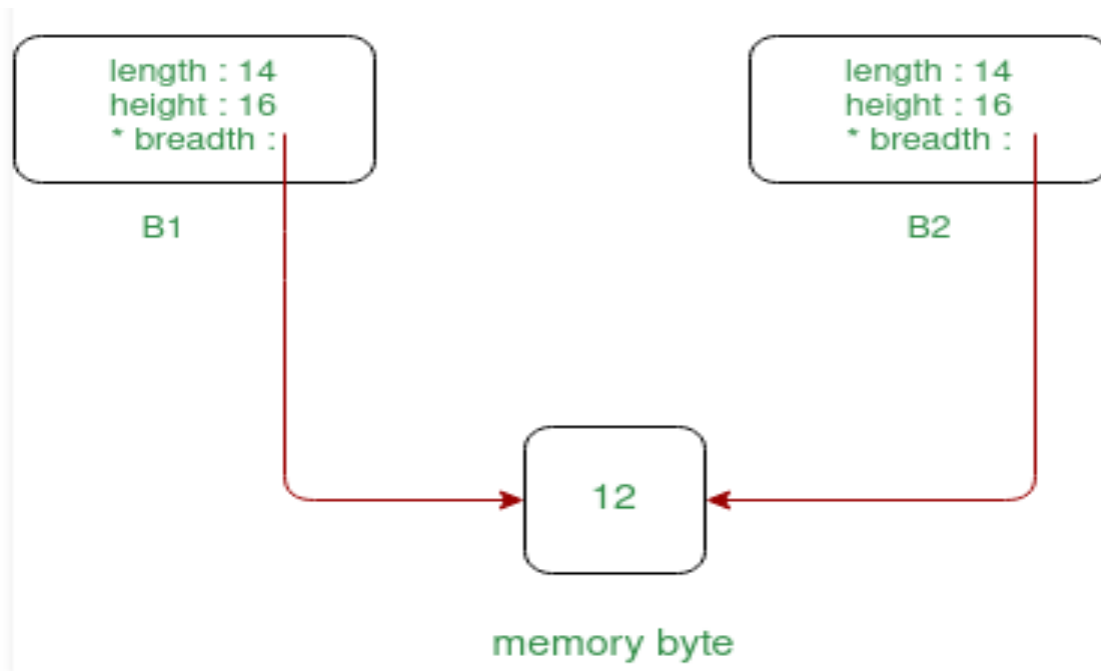
length : 14
breadth : 12
height : 16

B1

length : 14
breadth : 12
height : 16

B2

- **Shallow Copy** of object if some variables are defined in heap memory, then:



Deep Copy

- Deep copy is only possible with user-defined copy constructors
- In user-defined copy constructors, we make sure that pointers (or references) of copied object point to new memory locations

Deep copy

```
#include <iostream>
using namespace std;
class A{
    int val;
    int *ptr;
    public:
        A(int val1,int p) {
            val = val1;
            ptr = new int;
            *ptr = p;}
        void update() { val++; (*ptr)++;}
        void showVal() { cout << "Value: " << val << " ptr is "
            << *ptr << endl;
        }
        A(const A &other) {
            val = other.val;
            ptr = new int;
            *ptr= *(other.ptr);} // Allocate and copy the value
};
int main()
{
    A a1( 10,20);
    a1.showVal( );

    A a2(a1);

    a2.showVal();
    a2.update();

    a1.showVal( );
    a2.showVal( );

}
```

Deep Copy

```
#include <iostream>
using namespace std;
class A{
    int val;
    int *ptr;
    public:
        A(int val1,int p) {
            val = val1;
            ptr = new int;
            *ptr = p;}
        void update() { val++; (*ptr)++;}
        void showVal() { cout << "Value: " << val << " ptr is "
            << *ptr << endl;
        }
        A(const A &other) {
            val = other.val;
            ptr = new int{*(other.ptr)}; } // Allocate and copy the value
};
int main()
{
    A a1( 10,20 );
    a1.showVal( );

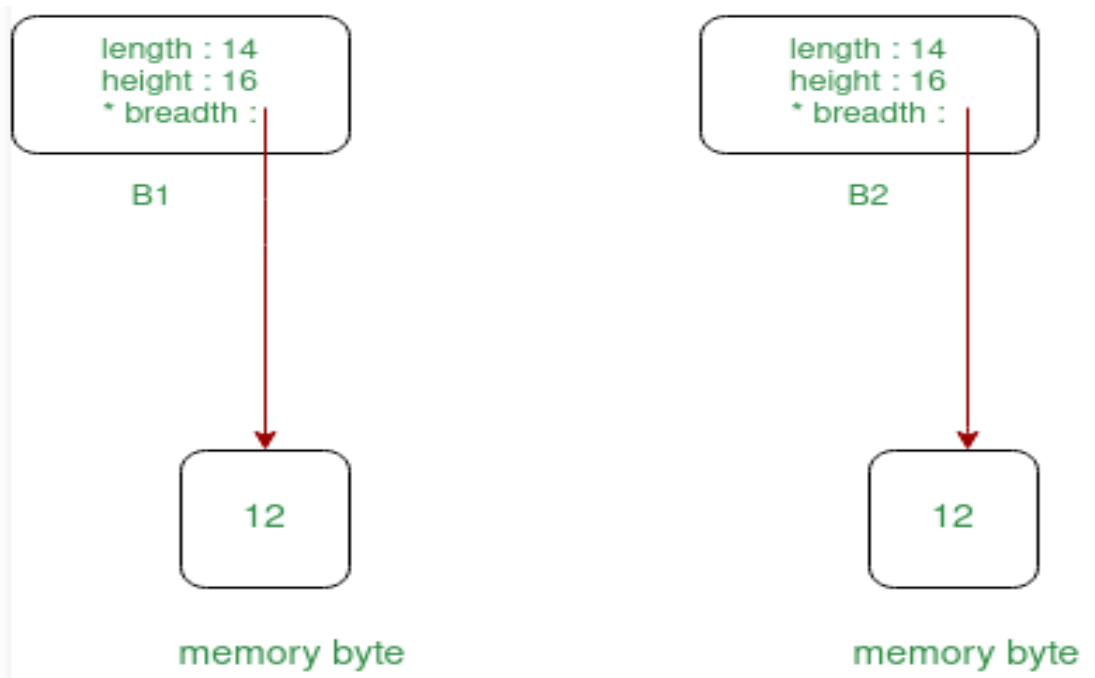
    A a2 = a1;

    a2.showVal();
    a2.update();

    a1.showVal( );
    a2.showVal( );

}
```

Deep Copy



Another example

- `#include <iostream>`
- `using namespace std;`
- `class box {`
- `int length;`
- `int* breadth;`
- `int height;`
- `public:`
- `box() {`
- `breadth = new int; }`
- `void set_dimension(int len, int brea, int heig) {`
- `length = len;`
- `*breadth = brea;`
- `height = heig;             }`
- `void show_data() {`
- `cout << " Length = " << length`
- `<< "\n Breadth = " << *breadth`
- `<< "\n Height = " << height`
- `<< endl; }`

Another example

- `box(box& sample)`
- `{`
- `length = sample.length;`
- `breadth = new int;`
- `*breadth = *(sample.breadth);`
- `height = sample.height;`
- `}`
- `// Destructors`
- `~box()`
- `{`
- `delete breadth;`
- `}`
- `};`

Another example

- `// Driver Code`
- `int main()`
- `{`
- `// Object of class first`
- `box first;`
- `// Set the dimensions`
- `first.set_dimension(12, 14, 16);`
- `// Display the dimensions`
- `first.show_data();`
- `box second = first;`
- `// Display the dimensions`
- `second.show_data();`
- `return 0;`
- `}`

Why copy constructor?

- If we remove copy constructor from the above program, we don't get the expected output
- The changes made to second reflect in first as well which is never expected.

Discussion

- Can we make copy constructor private?
- Why argument to a copy constructor must be passed as a reference?
- Why argument to a copy constructor should be const?

Can we make copy constructor private

- Yes, you can make a copy constructor private member in C++. Making so, you are restricting someone copying your object during object creation.
- objects of that class become non-copyable
- A common reason to make copy constructor private is to disable default implementation of these operations.

Why argument to a copy constructor must be passed as a reference?

- It is very essential to pass objects as reference. If an object is passed as value to the Copy Constructor then its copy constructor would call itself, to copy the actual parameter to the formal parameter. Thus an endless chain of call to the copy constructor will be initiated. This process would go on until the system runs out of memory.

```
class MyClass {  
public:  
    MyClass(MyClass obj) { // Incorrect: passing by value  
        std::cout << "Copy constructor called" << std::endl;  
    }  
};
```

```
int main() {  
    MyClass obj1; // This line will cause infinite recursion (if  
allowed)  
    MyClass obj2 = obj1; // Calls the copy constructor  
}
```

Why argument to a copy constructor should be const?

- we should use const in C++ wherever possible so that objects are not accidentally modified